

Docker Inside Jenkins Agents, Docker Plugins, Image Publishing, GitHub Integration, Backup/Restore and Pipeline Best Practices

Complete Step-by-Step Practical Guide for Students

This document is written for beginners. Follow each command one by one. Do not skip steps. Replace sample usernames, repository names, and tokens with your own values.

1. Learning Objectives

- Run Jenkins using Docker.
- Understand Docker inside Jenkins agents.
- Use Docker Pipeline and Docker plugins.
- Build Docker images from Jenkins.
- Publish images to Docker Hub.
- Publish images to GitHub Container Registry (GHCR).
- Connect Jenkins with GitHub using webhook and multibranch pipeline.
- Backup and restore Jenkins.
- Write clean and safe Jenkins pipelines.

2. Required Software and Accounts

Requirement	Purpose
Git	To push project code to GitHub
Docker Desktop or Docker Engine	To build and run containers
Jenkins	To run CI/CD pipeline
GitHub account	To host source code
Docker Hub account	To publish Docker images
Text editor	VS Code or any editor

3. Basic Architecture

```
Student pushes code to GitHub
GitHub sends webhook to Jenkins
Jenkins checks out source code
Jenkins builds Docker image
Jenkins tests Docker image
Jenkins logs in to Docker Hub or GHCR
Jenkins pushes Docker image
Image becomes ready for deployment
```

4. Check System Prerequisites

Step 1: Check Git

```
git --version
```

Expected result: Git version should be displayed.

Step 2: Check Docker

```
docker --version
```

Expected result: Docker version should be displayed.

Step 3: Check Docker is running

```
docker ps
```

Expected result: No error should appear. If Docker is not running, start Docker Desktop or Docker service.

5. Start Jenkins Using Docker

Step 1: Create Jenkins data volume

```
docker volume create jenkins_home
```

This volume stores Jenkins jobs, plugins, credentials, and configuration.

Step 2: Run Jenkins container

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:ls
```

Note: On Windows or macOS Docker Desktop, mounting /var/run/docker.sock usually works inside Linux-based Docker Desktop. If permission issues occur, use a Linux VM or WSL2 environment.

Step 3: Open Jenkins in browser

```
http://localhost:8080
```

Step 4: Get Jenkins initial admin password

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 5: Complete Jenkins setup

1. Paste the password in Jenkins setup page.
2. Click Install suggested plugins.
3. Create admin user.
4. Click Save and Finish.
5. Click Start using Jenkins.

6. Install Required Jenkins Plugins

Open Jenkins dashboard and go to Manage Jenkins -> Plugins -> Available plugins.

- Docker Pipeline
- Docker plugin
- Git
- GitHub

- GitHub Branch Source
- Pipeline
- Credentials Binding
- Workspace Cleanup

After installation, restart Jenkins if Jenkins asks for restart.

7. Create a Sample Application

Step 1: Create project folder

```
mkdir jenkins-docker-demo
cd jenkins-docker-demo
```

Step 2: Create app.py

```
cat > app.py <<'EOF'
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Jenkins Docker Pipeline!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF
```

Step 3: Create requirements.txt

```
cat > requirements.txt <<'EOF'
flask
EOF
```

Step 4: Create Dockerfile

```
cat > Dockerfile <<'EOF'
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
EOF
```

Step 5: Test Docker image locally

```
docker build -t jenkins-docker-demo .
docker run -p 5000:5000 jenkins-docker-demo
```

Open browser: <http://localhost:5000>

Stop the running container by pressing CTRL + C in terminal.

8. Push Project to GitHub

Step 1: Initialize Git

```
git init
```

Step 2: Add files

```
git add .
```

Step 3: Commit files

```
git commit -m "Initial Docker Jenkins demo"
```

Step 4: Rename branch to main

```
git branch -M main
```

Step 5: Add remote repository

```
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/jenkins-docker-demo.git
```

Step 6: Push code

```
git push -u origin main
```

Note: Replace YOUR_GITHUB_USERNAME with your real GitHub username.

9. Jenkins Pipeline: Build Docker Image

Step 1: Create Jenkinsfile

```
cat > Jenkinsfile <<'EOF'
pipeline {
    agent any

    environment {
        IMAGE_NAME = "jenkins-docker-demo"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {

```

```

        checkout scm
    }
}

stage('Build Docker Image') {
    steps {
        sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
        sh 'docker tag $IMAGE_NAME:$IMAGE_TAG $IMAGE_NAME:latest'
    }
}

stage('Test Docker Image') {
    steps {
        sh 'docker run --rm $IMAGE_NAME:$IMAGE_TAG python --version'
    }
}
}

post {
    success {
        echo 'Docker image build completed successfully.'
    }
    failure {
        echo 'Pipeline failed. Check console output.'
    }
}
}
EOF

```

Step 2: Push Jenkinsfile to GitHub

```

git add Jenkinsfile
git commit -m "Add Jenkins pipeline"
git push

```

Step 3: Create Jenkins pipeline job

6. Open Jenkins dashboard.
7. Click New Item.
8. Enter name: docker-build-demo.
9. Select Pipeline.
10. Click OK.
11. Go to Pipeline section.
12. Select Pipeline script from SCM.
13. Select Git.
14. Paste GitHub repository URL.
15. Set branch as main.
16. Set Script Path as Jenkinsfile.
17. Click Save.

18. Click Build Now.

10. Docker Inside Jenkins Agents

Docker agent means Jenkins runs a stage inside a Docker container. This is useful because Jenkins does not need Maven, Node, Python, or Java installed directly on the host.

Example: Run Maven inside Docker agent

```
pipeline {
  agent {
    docker {
      image 'maven:3.9.9-eclipse-temurin-17'
    }
  }

  stages {
    stage('Check Maven Version') {
      steps {
        sh 'mvn -version'
      }
    }
  }
}
```

Example: Run Python inside Docker agent

```
pipeline {
  agent {
    docker {
      image 'python:3.11-slim'
    }
  }

  stages {
    stage('Check Python') {
      steps {
        sh 'python --version'
        sh 'pip --version'
      }
    }
  }
}
```

11. Docker Hub Publishing Practical

Step 1: Create Docker Hub access token

19. Login to Docker Hub.
20. Open Account Settings.
21. Open Security.
22. Create New Access Token.
23. Copy the token safely.

Step 2: Add Docker Hub credential in Jenkins

24. Go to Manage Jenkins.
25. Click Credentials.
26. Open Global credentials.
27. Click Add Credentials.
28. Select Username with password.
29. Username: your Docker Hub username.
30. Password: Docker Hub access token.
31. ID: dockerhub-credentials.
32. Click Create.

Step 3: Jenkinsfile for Docker Hub push

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/jenkins-docker-demo"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
                sh 'docker tag $IMAGE_NAME:$IMAGE_TAG $IMAGE_NAME:latest'
            }
        }

        stage('Docker Hub Login') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-credentials',
```

```

        usernameVariable: 'DOCKER_USER',
        passwordVariable: 'DOCKER_PASS'
    )) {
        sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
    }
}

stage('Push Image to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
        sh 'docker push $IMAGE_NAME:latest'
    }
}

post {
    always {
        sh 'docker logout || true'
    }
}
}

```

12. GitHub Container Registry GHCR Publishing Practical

Step 1: Create GitHub token

33. Open GitHub.
34. Go to Settings.
35. Open Developer settings.
36. Open Personal access tokens.
37. Create token.
38. Enable write:packages.
39. Enable read:packages.
40. Enable repo if repository is private.
41. Copy the token safely.

Step 2: Add GHCR credential in Jenkins

42. Go to Manage Jenkins -> Credentials.
43. Add new credential.
44. Kind: Username with password.
45. Username: GitHub username.
46. Password: GitHub token.
47. ID: github-ghcr-token.
48. Click Create.

Step 3: Jenkinsfile for GHCR push

```

pipeline {

```

```
agent any

environment {
    GHCR_IMAGE = "ghcr.io/YOUR_GITHUB_USERNAME/jenkins-docker-demo"
    IMAGE_TAG = "${BUILD_NUMBER}"
}

stages {
    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Build Docker Image') {
        steps {
            sh 'docker build -t $GHCR_IMAGE:$IMAGE_TAG .'
            sh 'docker tag $GHCR_IMAGE:$IMAGE_TAG $GHCR_IMAGE:latest'
        }
    }

    stage('Login to GHCR') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'github-ghcr-token',
                usernameVariable: 'GHCR_USER',
                passwordVariable: 'GHCR_TOKEN'
            )]) {
                sh 'echo $GHCR_TOKEN | docker login ghcr.io -u $GHCR_USER --password-stdin'
            }
        }
    }

    stage('Push Image to GHCR') {
        steps {
            sh 'docker push $GHCR_IMAGE:$IMAGE_TAG'
            sh 'docker push $GHCR_IMAGE:latest'
        }
    }
}

post {
    always {
        sh 'docker logout ghcr.io || true'
    }
}
```

```
}  
}  
}
```

13. Jenkins and GitHub Integration

Step 1: Install GitHub-related plugins

- Git plugin
- GitHub plugin
- GitHub Branch Source plugin

Step 2: Create GitHub token credential

49. Create GitHub personal access token.
50. Go to Jenkins Credentials.
51. Add credential as Secret text.
52. ID: github-token.
53. Secret: paste GitHub token.

Step 3: Create Multibranch Pipeline

54. Click New Item.
55. Enter name: github-multibranch-demo.
56. Select Multibranch Pipeline.
57. Click OK.
58. In Branch Sources, select GitHub.
59. Add GitHub credential.
60. Enter repository owner.
61. Select repository.
62. Save.

Step 4: Add GitHub webhook

63. Open your GitHub repository.
64. Go to Settings.
65. Open Webhooks.
66. Click Add webhook.
67. Payload URL: `http://YOUR_JENKINS_URL/github-webhook/`.
68. Content type: `application/json`.
69. Event: Just the push event.
70. Click Add webhook.

14. Backup Jenkins

Step 1: Stop Jenkins container

```
docker stop jenkins
```

Step 2: Create backup file

```
docker run --rm \  
-v jenkins_home:/var/jenkins_home \  
-v $(pwd):/backup \  
ubuntu tar czf /backup/jenkins-backup.tar.gz /var/jenkins_home
```

Step 3: Start Jenkins again

```
docker start jenkins
```

Step 4: Verify backup file

```
ls -lh jenkins-backup.tar.gz
```

15. Restore Jenkins

Step 1: Stop Jenkins

```
docker stop jenkins
```

Step 2: Restore backup

```
docker run --rm \
  -v jenkins_home:/var/jenkins_home \
  -v $(pwd):/backup \
  ubuntu tar xzf /backup/jenkins-backup.tar.gz -C /
```

Step 3: Start Jenkins

```
docker start jenkins
```

Step 4: Open Jenkins

```
http://localhost:8080
```

Your jobs, plugins, credentials, and configuration should be restored.

16. Complete Best-Practice Jenkinsfile

```
pipeline {
  agent any

  options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }

  environment {
    IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/jenkins-docker-demo"
    IMAGE_TAG = "${BUILD_NUMBER}"
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
  }
}
```

```

}

stage('Build Docker Image') {
    steps {
        sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
    }
}

stage('Test Docker Image') {
    steps {
        sh 'docker run --rm $IMAGE_NAME:$IMAGE_TAG python --version'
    }
}

stage('Push Docker Image') {
    when {
        branch 'main'
    }
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            sh
            echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
            docker tag $IMAGE_NAME:$IMAGE_TAG $IMAGE_NAME:latest
            docker push $IMAGE_NAME:$IMAGE_TAG
            docker push $IMAGE_NAME:latest
            docker logout
        }
    }
}

post {
    success {
        echo 'Pipeline completed successfully.'
    }
    failure {
        echo 'Pipeline failed.'
    }
    always {

```

```

        cleanWs()
    }
}
}

```

17. Pipeline Best Practices

- Do not write passwords in Jenkinsfile.
- Store secrets in Jenkins Credentials Manager.
- Use access tokens instead of account passwords.
- Use Docker tags such as BUILD_NUMBER, latest, Git commit hash, or version number.
- Push images only from main branch.
- Use Multibranch Pipeline for real projects.
- Use webhooks instead of manual builds.
- Clean workspace after pipeline execution.
- Backup Jenkins regularly.
- Use Docker agents to avoid installing tools manually on Jenkins machine.
- Keep Jenkinsfile in GitHub repository.
- Keep Dockerfile simple and small.
- Use .dockerignore to reduce build context size.

18. Create .dockerignore File

Create .dockerignore to prevent unnecessary files from entering Docker build context.

```

cat > .dockerignore <<'EOF'
.git
.gitignore
Jenkinsfile
*.log
__pycache__
*.pyc
.env
EOF

```

19. Common Errors and Solutions

Error	Solution
permission denied while connecting to Docker socket	Jenkins user cannot access Docker socket. Use proper Docker permissions or run Jenkins with mounted docker.sock carefully.
docker: command not found	Docker CLI is not available inside Jenkins container. Install Docker CLI in Jenkins image or use host/agent with Docker.
denied: requested access to the resource is denied	Wrong Docker Hub username, repository name, or login credential.
unauthorized: authentication required	Credential ID/token is wrong or expired.
GitHub webhook not triggering	Check Jenkins public URL, webhook payload URL, firewall, and GitHub webhook delivery logs.
Jenkinsfile not found	Check branch name and Script Path. Usually Script Path should be Jenkinsfile.

20. Student Practice Tasks

71. Create GitHub repository named jenkins-docker-demo.
72. Create Python Flask app.
73. Create Dockerfile.
74. Build Docker image locally.
75. Run Docker container locally.
76. Create Jenkinsfile.
77. Create Jenkins pipeline job.
78. Build image using Jenkins.
79. Push image to Docker Hub.
80. Push image to GHCR.
81. Create GitHub webhook.
82. Modify app.py and push code.
83. Verify Jenkins starts automatically.
84. Backup Jenkins.
85. Restore Jenkins backup.

21. Viva Questions

- What is Jenkins?
- What is Docker?
- Why do we use Docker inside Jenkins agents?
- What is Docker Hub?
- What is GHCR?
- What is Jenkinsfile?
- What is a webhook?
- Why should credentials not be written inside Jenkinsfile?
- What is a multibranch pipeline?
- Why is Jenkins backup important?

22. Final Summary

In this practical, students learned how to use Jenkins with Docker. They created a sample application, wrote a Dockerfile, built Docker images through Jenkins, published images to Docker Hub and GitHub Container Registry, connected Jenkins with GitHub using webhook, and practiced backup and restore. This workflow is a real DevOps CI/CD pipeline used in software delivery.

23. References

- Jenkins official Docker Pipeline documentation: <https://www.jenkins.io/doc/book/pipeline/docker/>
- Jenkins Docker Pipeline plugin documentation: <https://plugins.jenkins.io/docker-workflow/>
- Jenkins GitHub Branch Source plugin documentation: <https://plugins.jenkins.io/github-branch-source/>
- GitHub Container Registry documentation: <https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry>